mapped route gets looked up, the request gets passed through multiple middleware processors (authentication, session management, CSRF handling, and more), and then finally your code gets run. And of course, your code is Python, which is not the speediest language out there. Static content, like CSS files, site images, and JavaScript, shouldn't be served by Django; they don't need all that extra overhead. Likewise, any user-uploaded content also doesn't typically need all the mechanics your view requires.

Django is invoked through either a WSGI or ASGI interface, with a web server responsible for handling the actual call and passing it down to Django. Therefore, when you go to production, you need a web server. With such a beast in place, it makes a lot of sense to have the static and uploaded files served directly, skipping all that Django overhead. Figure B.1 shows the typical architecture with a web server fronting Django and directly hosting the static and uploaded files.

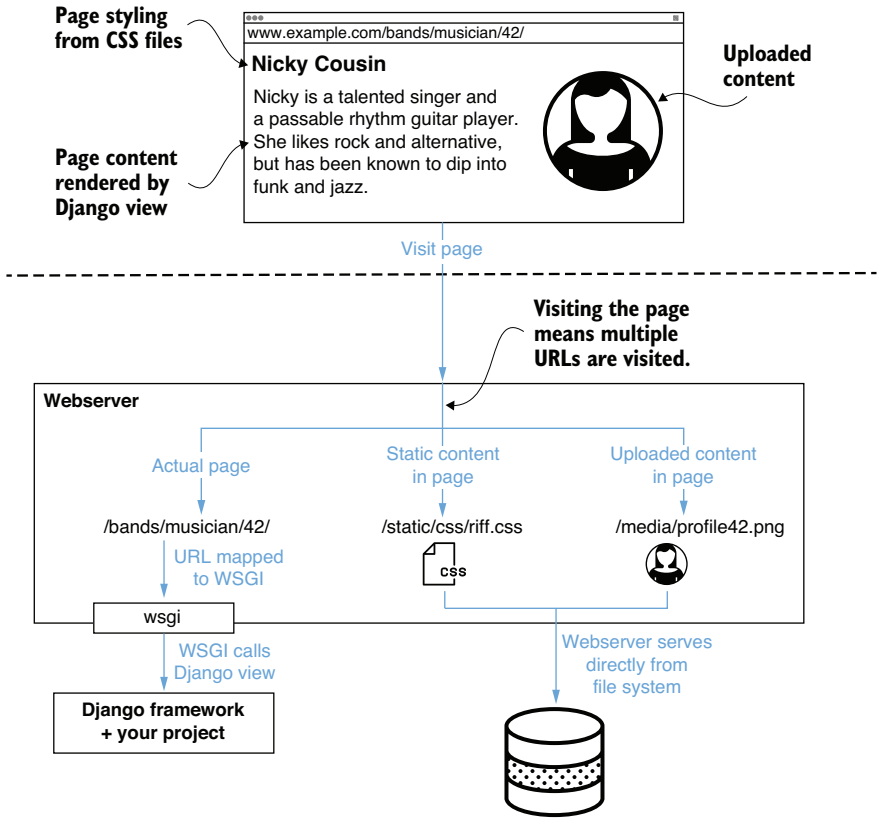


Figure B.1 Django through WSGI and other files served directly